AEP FROM ZERO • PART 1 • KNOW YOUR CUSTOMER (AEP)

EPISODE 03

XDM Schema Explained: Why Orders Are Events

XDM schema explained from zero: what the Experience Data Model (XDM) is, and why every Adobe Experience Platform (AEP) project starts with it. In episode 3 of AEP From Zero, Bloom's team designs its first XDM schemas on paper: an XDM Individual Profile schema for the customer and XDM ExperienceEvent schemas for web events and orders, built from field groups, with custom fields under the _bloom tenant namespace. You will learn the profile vs event test, why every event needs _id and

Previously: two worlds, both still empty

- Last time: the lake, the profile store, one checkbox
- Both worlds are still empty
- Today: XDM, schemas, classes, field groups
- Profile or event? One analogy, one test
- Priya's evening, read in three files

KEY IDEA

Before any data arrives, Bloom must agree what a customer is. Today: XDM, schemas, classes, data types, and Priya's evening in three files.

One email, three names

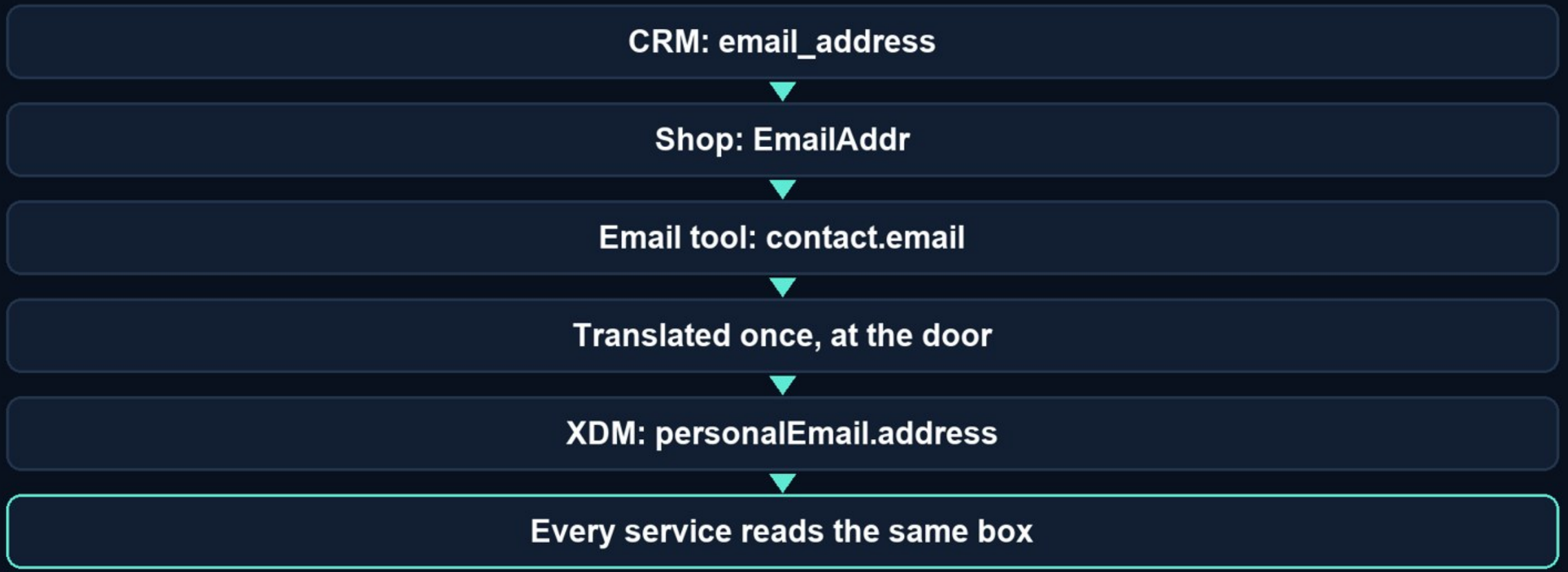
System	What it calls the customer's email
CRM export	email_address
E-commerce platform	EmailAddr
Email tool	contact.email

One fact, three names, three translations to maintain

KEY IDEA

Bloom's three systems name the same email three ways. Without one agreed definition, every audience, journey and report inherits the mess.

What is XDM? The shipping container



KEY IDEA

XDM, the Experience Data Model, is Adobe's one standard vocabulary for customer data: agree on the box once, and every service can carry it.

Schema, class and field groups

Ingredient	What it decides	Bloom example
Schema	Blueprint for one record kind	Bloom Customer Profile
Class	What the record is	XDM Individual Profile
Field groups	Bundles of fields snapped on	Personal Contact Details

KEY IDEA

A schema is the blueprint for one kind of record, built from a class plus field groups. The class is permanent once the schema is saved.

Bloom's three schemas, on paper

Schema	Class	Field groups
Bloom Customer Profile	XDM Individual Profile	Contact, Demographic, _bloom
Bloom Web Event	XDM ExperienceEvent	Consumer Experience Event
Bloom Order Event	XDM ExperienceEvent	Commerce Details, _bloom

Behaviour: Record for the profile, Time-series for both events

KEY IDEA

One profile schema for who the customer is, and two event schemas for what she does: web events and orders, with Bloom's own fields under _bloom.

Profile or event? The bank test

Profile: XDM Individual Profile	Event: XDM ExperienceEvent
Changes in place	Written once, never changes
Who is this person now?	What has this person done?
Tier, consent, home city	Orders, views, add-to-carts
New phone replaces old phone	A refund is a new event

KEY IDEA

Like a bank: account details change in place, transactions are written once and never change. Profile answers who; events answer what was done.

The litmus test: can it be recomputed?

Bloom field	Recomputed from events?	Where it lives
Email consent	No, a fact about her	Profile
Loyalty tier	No, Bloom's current claim	Profile
Order total	It is part of the order	Order event
Total spent this year	Yes, add up the orders	Not stored: compute it

KEY IDEA

For any hard field ask: can this be recomputed from events? If yes, compute it, never store it. If no, it is a fact that belongs on the profile.

_id and timestamp: what makes an event

- _id: a unique identifier for each event
- timestamp: the moment it happened
- No when? Then it is not an event
- Priya's add-to-cart: 7:44 PM, 18 August
- Episode 6: bad timestamps raise real errors

KEY IDEA

Every ExperienceEvent carries _id, a unique identifier, and timestamp, the moment it happened. Without a when, it is not an event at all.

Six data types every field carries

Type	Plain meaning	Priya example
string	Text	First name: Priya
integer	Whole number	Points: 2,340
date-time	An exact moment	7:44 PM, 18 August 2026
boolean	True or false	Email consent: true
object, array	A bundle, a list	Line items in her order

KEY IDEA

Every field has a data type. The series names six: string, integer, date-time, boolean, object and array. The type decides what the platform accepts.

_bloom: Bloom's own space in the schema

- Adobe's standard fields sit at the root
- Bloom's own fields sit under _bloom
- The editor shows it as _{tenantId}
- Adobe's changes never collide with yours
- Reuse a standard field before inventing one

KEY IDEA

Custom fields live under the tenant namespace, _bloom at Bloom, so Adobe's evolving standard fields can never collide with Bloom's own.

Why you can't delete a schema field

Change	Once data has landed
Add a new field	Allowed
Remove a field	Not allowed
Change a field's type	Not allowed
Change the class	Never: fixed once saved

Design on paper, check against samples, then build

KEY IDEA

Once data has landed, schema changes are additive-only: add fields, never remove or retype them. Like adding rooms to a house people live in.

Priya slowly: the profile record

Profile record field	Priya's value
Name	Priya Sharma
personalEmail.address	priya.sharma@example.com
_bloom loyalty	Gold, 2,340 points, joined 2 Nov 2023
_bloom consent	Email true, SMS false
identityMap	Values that identify her (episode 4)

KEY IDEA

The profile record is Bloom's current claim about who Priya is: name, email, Gold tier, 2,340 points, consent. It knows nothing about what she did.

7:44 PM: an add-to-cart with no name

Web event field	Value
_id	evt-web-20260818-000341
timestamp	18 Aug 2026, 7:44 PM
eventType	commerce.productListAdds
identityMap	ECID 384729198620114567 43 only
productListItems	BLM-SER-VITC-30 at 34.00

KEY IDEA

Priya's web event knows exactly what happened, an add-to-cart for the Vitamin C Serum, but carries only an anonymous ECID: no name, no email.

7:52 PM: the order, and what it knows

Order event field	Value
_id	evt-order-20260818-088412
identityMap	Email and BloomID
commerce.order	BLM-88412, total 54.00
Line item 1	BLM-SER-VITC-30 at 34.00
Line item 2	BLM-MOIST-HYA-50 at 20.00

KEY IDEA

The order knows the person, the products, the money and the moment: email and BloomID, two line items, 54.00 in total, never edited afterwards.

Two uncomfortable truths: no cart, two Priyas

7:44 PM: add to cart, ECID only

▼

7:52 PM: order, email and BloomID

▼

No cart object stored anywhere

▼

Cart = adds not followed by a purchase

▼

Today: two Priyas, nothing links them

KEY IDEA

A cart is derived, not stored: add-to-cart events not followed by a purchase. And until identities link, the platform would see two Priyas.

Interview corner: why orders are events

- Orders are events: they never change
- Loyalty tier is profile: it changes in place
- Recomputable from events? Compute it
- Bad schemas pass their mistakes downstream

KEY IDEA

Orders are events because they never change. Loyalty tier is profile because it changes in place.

Recap, your question, and what comes next

- XDM: one box every service can carry
- Schema = class + field groups; class is fixed
- Recomputable from events? Then compute it
- Your question: store date of last order?
- Next: build the three schemas for real

KEY IDEA

XDM is the box, a schema is a class plus field groups, events never change, and the cart is derived. Next: building the schemas for real.